Stage 1 User Guide — Final (ArduPilot SITL + AirSim)

Oct 17, 2026 · @lukas

Stage 1 teaches the network to hover on one spot and come back when nudged off it, by copying a simple teacher. Run 1 (GPS mode) passed every check: transfer efficiency 98.85% (position), success 100%.

At a glance

A simple teacher flies the drone to targets near one home spot; the network learns to copy it. Train in GPS mode, rehearse in optical-flow mode, then fly the real drone.

Step	Where	Position comes from	Purpose
1. Train	SITL + AirSim, GPS mode	GPS (exact, no drift)	Collect clean data, train the network, measure results for the thesis
2. Check the sensor	Real drone, by hand (PosHold)	MTF-01 optical flow + rangefinder	Confirm the real flow sensor reads distance and direction correctly before the network flies (the SITL optical-flow model is not realistic here)
3. Real flight	Real drone indoors	MTF-01 optical flow + rangefinder	Final proof: hover, then the fan test (Stage 2)

Why train in GPS mode: optical-flow position is added up from small movements, so its error grows over a long session. In a 50-minute collection the drone slid into the wall while ArduPilot still thought it was on the spot. GPS has no such drift. The network sees the same 13 numbers either way, so a GPS-trained network runs unchanged with optical flow.

Part	What it is
Teacher	Speed = gain x distance to target; sideways gain 0.8 /s capped at 1.0 m/s, up/down gain 1.0 /s capped at 0.5 m/s. No hidden memory, so the network can copy it

Part	What it is
Network input (13)	Position minus target (3), velocity (3), attitude quaternion (4), body rates (3)
Network output (3)	Velocity command north, east, down (m/s), sent in GUIDED mode
Control rate	20 Hz, fixed (50 ms per step)
Network	13 -> 256 -> 256 -> 128 -> 3, ReLU, 102,659 parameters; loads directly into Stage 2 PPO
Targets	Within +/-1.0 m of home sideways, 2.0 +/- 0.5 m high
Safety	Stops if height < 0.3 m, tilt > 45 deg, or more than 2.5 m from home; lands automatically at the end

Files and folders

Ten files sit in the `stage_1` folder on Windows. Run every command from that folder.

File	Job
`ap_link.py`	Talks to ArduPilot (same file for SITL and the real drone): takeoff, settle, velocity commands, auto-landing
`pid_expert.py`	The teacher
`collect_demonstration.py`	Step 1: teacher flies, data is saved; makes a new run folder
`train_imitation.py`	Step 2: trains the network, checks it loads into PPO
`test_hover_policy.py`	Step 3: the network flies alone
`collect_stage1_statistics.py`	Step 4: teacher and network fly the same 10 targets
`rmse_baseline.py`	Step 5: RMSE tables and transfer efficiency
`vd_distribution.py`	Step 6: command histogram, teacher vs network
`make_stage1_summary.py`	Step 7: executive summary with PASS/CHECK
`sitl_optflow.parm`	Optical-flow settings for SITL (rehearsal only)

Every full run gets its own numbered folder, made automatically. The highest number is the latest.

```
stage_1/
├── (the 10 files)
└── runs/
    ├── practice/                quick tests (--run practice)
    ├── run1/                    first full run
    └── run2/ ...                next full collection = next number
        ├── demonstrations/
        │   └── expert_demonstrations_<sensor>_<date>.pkl / .csv / .json
        ├── models/
        │   ├── hover_policy_best.pth      trained network
        │   ├── obs_norm.npz               input scaling (Stage 2 reuses it)
        │   ├── stage1_ppo_init.zip        starting point for Stage 2
        │   └── model_info.json            training numbers
        ├── logs/stage1/
        │   ├── training_<date>.csv        loss per epoch
        │   ├── loss_curve_<date>.png      loss plot
        │   └── tb_<date>/                 TensorBoard
        └── results/
            ├── test_hover_<date>.csv / .json / .pkl
            ├── statistics_summary_<date>.pkl / .json
            ├── statistics_report_<date>.txt
            ├── rmse_baseline_<date>.txt
            ├── vd_distribution_<date>.png
            └── STAGE1_SUMMARY_run<N>.md
```

Script	Uses which run folder	Pick another
`collect_demonstration.py`	Makes a NEW folder (highest number + 1)	`--run practice` for a quick test
All other scripts	The latest run	`--run 1`, `--run practice`

Every script prints `Run folder: runs\run<N>` at the start; check it. If a collection fails in the first minutes, delete that run folder so the numbers stay clean. Inside a run, scripts always use the newest file (the date in the name).

One-time setup

Do this once. Two SITL folders keep the two sensor setups apart: GPS in `~/sitl_gps`, optical flow in `~/ardupilot/Tools/autotest`. Each folder keeps its own saved settings (`eeprom.bin`).

1. **Install the Python packages** in the Windows `airsimcode` environment (Python 3.9):

```
pip install pymavlink
```

```
pip install stable-baselines3==2.3.2
```

```
pip install gymnasium==0.29.1
```

```
pip install tensorboard
```

2. **AirSim settings.** `Documents\AirSim\settings.json` must contain the sensors block below (your IPs and ports stay as they are). The `Distance1` sensor is the downward rangefinder; it is needed for the optical-flow rehearsal and does no harm in GPS mode. `"Z": 1.5` spawns the drone on the floor: in this map the AirSim start point is about 1.5 m above the floor (AirSim z on the ground was +1.55 m before, +0.08 m after). Check it with the drone on the floor: `py -c "import airsim;c=airsim.MultirotorClient();c.confirmConnection();print('ground z =',round(c.simGetVehiclePose().position.z_val,2))"` should print about 0.0; with a new map, set Z to the value it prints.

```
{
  "SettingsVersion": 1.2,
  "SimMode": "Multirotor",
  "Vehicles": {
    "Copter": {
      "VehicleType": "ArduCopter",
      "UseSerial": false,
      "LocalHostIp": "172.28.0.1",
      "UdpIp": "172.28.15.216",
      "UdpPort": 9003,
      "ControlPort": 9002,
      "X": 0, "Y": 0, "Z": 1.5,
      "Sensors": {
        "Imu":          { "SensorType": 2, "Enabled": true },
        "Gps":          { "SensorType": 3, "Enabled": true },
        "Barometer":    { "SensorType": 1, "Enabled": true },
        "Magnetometer": { "SensorType": 4, "Enabled": true },
        "Distance1": {
```

```
          "SensorType": 5,
          "Enabled": true,
          "Yaw": 0, "Pitch": -90, "Roll": 0,
          "MinDistance": 0.2,
          "MaxDistance": 10,
          "DrawDebugPoints": true,
          "ExternalController": true
        }
      }
    }
  }
}
```

3. **Make the GPS SITL folder** (Ubuntu). Start AirSim first, then:

```
mkdir -p ~/sitl_gps
```

```
cd ~/sitl_gps
```

4. **First start with `-w`, this one time only.** `-w` wipes the saved settings in the current folder back to ArduPilot's defaults, which have GPS on. Never use `-w` in `~/ardupilot/Tools/autotest`: it would erase the optical-flow settings.

```
~/ardupilot/Tools/autotest/sim_vehicle.py -v ArduCopter -f airsim-copter -w --console --map --sim-address=172.28.0.1 --out=127.0.0.1:14550
```

5. **Turn off arming checks** (SITL only) in the MAVProxy console:

```
param set ARMING_CHECK 0
```

6. **Restart instead of `reboot`:** stop AirSim, Ctrl+C in SITL, start AirSim, start SITL again without `-w` (Start of every session, below). `reboot` disconnects AirSim and does not always reconnect.
7. **Check the GPS-mode parameters** in the MAVProxy console. Each value must match.

Parameter	Expected value	Check command (MAVProxy)
`GPS_TYPE`	1 (GPS on)	`param show GPS_TYPE`
`EK3_SRC1_POSXY`	3 (GPS position)	`param show EK3_SRC1_POSXY`
`EK3_SRC1_VELXY`	3 (GPS velocity)	`param show EK3_SRC1_VELXY`
`EK3_SRC1_POSZ`	1 (barometer height)	`param show EK3_SRC1_POSZ`
`EK3_SRC1_VELZ`	3 (GPS vertical velocity)	`param show EK3_SRC1_VELZ`

Parameter	Expected value	Check command (MAVProxy)
`EK3_SRC1_YAW`	1 (compass)	`param show EK3_SRC1_YAW`
`GUID_OPTIONS`	0 (default)	`param show GUID_OPTIONS`
`ARMING_CHECK`	0 (arming checks off, SITL only)	`param show ARMING_CHECK`

The scripts read these values themselves: `EK3_SRC1_POSXY 3` means GPS mode (files named `_gps_`), `EK3_SRC1_VELXY 5` means optical-flow mode (files named `_optflow_`).

The optical-flow folder (`~/ardupilot/Tools/autotest`, loaded once with `param load sitl_optflow.parm`) is only needed for the flow diagnostic; how to load it and enter optical-flow mode is in "Entering optical-flow mode in SITL" below.

Start of every session (GPS mode)

Always AirSim first, then SITL, then the scripts.

1. **Start AirSim** (Windows). Put the drone in an open spot, at least 3 m from walls (safety fence is 2.5 m from home).
2. **Start SITL from the GPS folder** (Ubuntu), without `-w`:

```
cd ~/sitl_gps
```

```
~/ardupilot/Tools/autotest/sim_vehicle.py -v ArduCopter -f airsim-copter --
console --map --sim-address=172.28.0.1 --out=127.0.0.1:14550
```

3. **Check you are in GPS mode** (MAVProxy). It must show 3; if it shows 0 you are in the optical-flow folder: stop SITL and start again from `~/sitl_gps`.

```
param show EK3_SRC1_POSXY
```

4. **Wait for GPS lock**, about 30 s: the console shows `GPS: 3D Fix`.
5. **Open PowerShell** (Windows, `airsimcode` environment) in the `stage_1` folder. The scripts connect directly to `tcp:127.0.0.1:5762` (MAVProxy limits its own stream to 4 Hz, too slow for 20 Hz control).

You do not need to restart AirSim or SITL between scripts: each script takes off by itself and lands at the end. Restart both only if something goes wrong.

Practice run (about 2 minutes)

Always do this before a full run. It saves into `runs\practice`, so it does not use up a run number.

```
py .\collect_demonstration.py --episodes 5 --run practice
```

What good output looks like (your run 1, GPS mode):

```
Firmware: 4.4.0-dev | sensor mode: gps
[link] mode GUIDED_NOGPS
[link] armed
[link] climbing to 2.0 m (GUIDED_NOGPS)
[link] mode GUIDED
[link] holding still - waiting for EKF yaw alignment
[fc] EKF3 IMU0 MAG0 in-flight yaw alignment complete
[link] height 2.xx m - moving to 2.00 m
[link] settled at 2.0x m - ready to fly
Home: N -13.62 E -2.37 m | targets within +/-1.0 m of home, 2.0 +/- 0.5 m
high | safety fence 2.5 m
Episode    1/5 | start 1.02 m -> end 0.01 m | loop 50.0 ms | overruns 0 | 0.3
min
...
[link] landing (mode LAND)...
[link] landed and disarmed
```

Check	Good	If not
`sensor mode`	`gps`	Wrong SITL folder (see Start of every session)
Order of lines	`settled` comes before `Home:`	Old `ap_link.py`: replace it
`end` per episode	Below 0.3 m (run 1: 0.01 m)	See Troubleshooting
`loop`	50.0 ms, overruns 0	Close other programs on the PC
In AirSim	Drone moves less than about 1.5 m from where it took off	See Troubleshooting
End	`landed and disarmed`	Type `mode LAND` in MAVProxy

The drone moves only a little on purpose: each target is at most 1 m sideways and 0.5 m up or down from home. It reaches the target in about 3 s and holds it for the rest of the 15 s episode. Stage 1 teaches "nudged off, come back, hold"; bigger pushes come in Stage 2. In

GPS mode the yaw-alignment wait can take up to 15 s and print `no yaw-alignment message seen - continuing`: this is normal.

Full run (about 75 minutes)

Seven commands in order, all in GPS mode. Steps 2-7 use the newest run folder automatically.

Step	Time	Expected (run 1)	Pass rule
1. Collect	~55 min	60,000 samples, 200 episodes, end ~0.01 m, loop 50.0 ms	Loop within 5% of 50 ms, overruns <= 1%, teacher settled >= 80%
2. Train	~5 min	Val loss 0.0002, transfer check PASS	Transfer to PPO identical (difference < 1e-5)
3. Test	~4 min	10/10 success	Success >= 90%
4. Statistics	~8 min	Transfer efficiency 98.85% (position)	Transfer efficiency >= 90%
5-7. Tables, histogram, summary	~1 min	All PASS	-

1. **Collect data.** Makes the next run folder (`RUN1` the first time). The drone lands by itself at the end.

```
py .\collect_demonstration.py --episodes 200 --steps 300
```

2. **Train.** Check the last lines say `-> PASS`. The validation loss should keep falling and end far below 0.01.

```
py .\train_imitation.py
```

To see the loss curves, run this in a second PowerShell window, then open http://localhost:6006:

```
tensorboard --logdir runs
```

3. **Test the network alone.** Takes off, flies 10 fixed targets (same every time), lands.

```
py .\test_hover_policy.py --episodes 10
```

4. **Compare teacher and network.** The drone settles at home for 10 s, the teacher flies 10 targets, it settles at home again, the network flies the same 10 targets, then it lands. Both start from the same place, so the comparison is fair.

```
py .\collect_stage1_statistics.py --episodes 10
```

5. **RMSE tables and transfer efficiency.**

```
py .\rmse_baseline.py
```

6. **Command histogram** (vertical by default; `--axis vn` or `--axis ve` for north and east).

```
py .\vd_distribution.py
```

7. **Executive summary.** Reads only from the run folder and shows PASS/CHECK for steps 1-4.

```
py .\make_stage1_summary.py
```

To redo any step for an older run, add `--run <N>`, e.g. `py .\make_stage1_summary.py --run 1`. To stop a script early, press Ctrl+C: it saves what it has and lands.

Results: what to look at

Four numbers decide Stage 1; everything else supports them.

Result	Question it answers	Where	Good if
Transfer efficiency, position	Does the network fly as accurately as the teacher? (main thesis number)	`statistics_report`, `rmse_baseline`	>= 90%, and not above ~100%
Success rate	Does the network fly alone without failing?	`test_hover_<date>.json`	>= 90%
End error	How close does it hold the spot?	Test and statistics files	< 0.3 m
Loop time	Was the control rate steady?	Every script	50 ms, almost no overruns

Transfer efficiency = (1 - (BC_RMSE - PID_RMSE) / PID_RMSE) x 100%. Below 100% means the network is slightly less accurate than the teacher; 100% means equal.

Above 100% is a warning sign, not a good result. The network learned by copying, so it cannot really beat the teacher. A value clearly above 100% means the two did not get the same test conditions. In run 1 the first, unfair version gave 112% (position) and 146% (altitude): after takeoff the drone overshot to 4.0 m, so the teacher started episode 1 from 4.0 m while the network started from 2.0 m. After the fix (settle at 2.0 m and at home before each controller) it gave 98.85% and 97.32%. To check the start of episode 1 for both controllers:

```
py -c "import
pickle,glob;f=sorted(glob.glob('runs/run1/results/statistics_summary_*.pkl'))
[-1];d=pickle.load(open(f,'rb'));[print(c,'ep1 start
error',round(d['episodes'][c][0]['pos_err'][0],2),'m | start
height',round(d['episodes'][c][0]['alt'][0],2),'m | target
height',round(d['episodes'][c][0]['target_alt'],2),'m') for c in
('pid','bc')]"
```

Both lines should show a start height near 2.0 m and a similar start error.

Two transfer efficiencies. Position uses the full distance to the target (sideways and height together) and is the main number. Altitude uses height only; it is reported separately because height (thrust) and sideways movement (tilt) are controlled differently, so a weakness in one shows clearly.

File (in `runs\run<N>\`)	What it shows
`demonstrations/..._<date>.csv`	One line per collected episode: start and end error, RMSE, settled
`demonstrations/..._<date>.json`	Collection settings, firmware, parameters, teacher, timing
`models/model_info.json`	Training numbers: samples, split, best validation loss and epoch, RMSE per axis, transfer check
`logs/stage1/loss_curve_<date>.png`	Training and validation loss per epoch (Graph 1)
`results/test_hover_<date>.csv / .json`	Network alone: per-episode errors, success and settled rates
`results/statistics_report_<date>.txt`	Teacher vs network tables, readable
`results/statistics_summary_<date>.pkl`	Every step of every episode, for graphs
`results/rmse_baseline_<date>.txt`	Aggregate and per-episode RMSE, transfer efficiency

File (in `runs\run<N>\`)	What it shows
`results/vd_distribution_<date>.png`	Command histograms, teacher vs network on the same bins
`results/STAGE1_SUMMARY_run<N>.md`	Executive summary, PASS/CHECK per step, readiness for Stage 2

The statistics script also stores, per episode, the target and the error at every step (`episodes` -> `pid` / `bc` in the `.pkl`), so later graphs (height over time, RMSE per episode, maximum drift) can be drawn without flying again.

Optical flow in simulation: finding and real-drone check

The SITL optical-flow model is not realistic in this AirSim setup, so there is no optical-flow rehearsal in simulation. `flow_diagnostic.py` (a one-off check, not part of the pipeline) compared ArduPilot's estimate with AirSim's true position, same code and drone in both modes:

Check	GPS mode	Optical-flow mode
Hold still 40 s: real movement	0.01 m	0.33 m
Hold still 40 s: gap estimate vs real	0.01 m	0.25 m
Distance ArduPilot / real distance (0.5 m/s steps)	0.90 (all 4 directions)	0.39
Direction error	1-5 deg	0-3 deg
Verdict	Commands work	Speed scaled wrong, drone drifts

What this means:

- The code and commands are correct: in GPS mode the drone goes where it is told and holds still.
- In optical-flow mode ArduPilot sees only 39% of the real movement, so the drone flies too far and slides away. The teacher and the trained network both failed there for this reason.
- It is a simulator problem. SITL builds its fake flow reading from AirSim's GPS speed (ArduPilot source `SIM_AirSim.cpp`); fixing the start height (below) did not change it. The real MTF-01 measures the real floor, so it does not have this problem.

Thesis wording: "The SITL optical-flow model did not reproduce real sensor behaviour in this setup (ArduPilot estimated 39% of true displacement, vs 90% with GPS). The policy was therefore trained with GPS-quality state and validated directly on hardware."

Entering optical-flow mode in SITL

Optical-flow settings live in their own folder, `~/ardupilot/Tools/autotest`, separate from GPS (`~/sitl_gps`). Each folder keeps its own saved settings (`eeprom.bin`), so loading here never touches GPS mode.

First time only (load the parameters):

1. Copy `sitl_optflow.parm` from `stage_1` (Windows) into `~/ardupilot/Tools/autotest` (Ubuntu).
2. Start AirSim.
3. Start SITL from the optical-flow folder, **without** `-w` (`-w` would wipe these settings):

```
cd ~/ardupilot/Tools/autotest
```

```
sim_vehicle.py -v ArduCopter -f airsim-copter --console --map --sim-address=172.28.0.1 --out=127.0.0.1:14550
```

4. In the MAVProxy console, load the optical-flow parameters:

```
param load sitl_optflow.parm
```

5. Turn off arming checks (SITL only):

```
param set ARMING_CHECK 0
```

6. Restart instead of `reboot`: close AirSim, Ctrl+C in SITL, start AirSim, start SITL again (step 3). The settings are now saved in this folder.
7. Check every parameter; each value must match.

Parameter	Expected value	Check command (MAVProxy)
`GPS_TYPE`	0 (GPS off)	`param show GPS_TYPE`
`EK3_SRC1_VELXY`	5 (optical flow)	`param show EK3_SRC1_VELXY`
`EK3_SRC1_POSXY`	0 (no absolute horizontal position)	`param show EK3_SRC1_POSXY`
`EK3_SRC1_POSZ`	2 (rangefinder height)	`param show EK3_SRC1_POSZ`
`EK3_SRC1_VELZ`	0 (none)	`param show EK3_SRC1_VELZ`

Parameter	Expected value	Check command (MAVProxy)
`EK3_SRC1_YAW`	1 (compass)	`param show EK3_SRC1_YAW`
`EK3_SRC_OPTIONS`	0 (flow-only setup)	`param show EK3_SRC_OPTIONS`
`SIM_FLOW_ENABLE`	1 (simulated optical flow on)	`param show SIM_FLOW_ENABLE`
`FLOW_TYPE`	10 (SITL flow sensor)	`param show FLOW_TYPE`
`RNGFND1_TYPE`	100 (AirSim distance sensor)	`param show RNGFND1_TYPE`
`RNGFND1_ORIENT`	25 (pointing down)	`param show RNGFND1_ORIENT`
`RNGFND1_MIN_CM`	20 (0.2 m)	`param show RNGFND1_MIN_CM`
`RNGFND1_MAX_CM`	1000 (10 m)	`param show RNGFND1_MAX_CM`
`ARMING_CHECK`	0 (arming checks off, SITL only)	`param show ARMING_CHECK`

Every time after that (enter optical-flow mode):

1. Start AirSim.
2. Start SITL from the optical-flow folder (step 3 above, no `-w`).
3. Confirm you are in optical-flow mode; it must show 5:

```
param show EK3_SRC1_VELXY
```

There is no GPS lock to wait for. The scripts take off in GUIDED_NOGPS, wait for a position from the flow sensor, then switch to GUIDED; the header shows `sensor mode: optflow` and files are named `_optflow_`.

Back to GPS mode: close AirSim and SITL, start AirSim, start SITL from `~/sitl_gps` (Start of every session); `param show EK3_SRC1_POSXY` must show 3.

To re-run the check (about 2 minutes, takes off, moves within about 1.5 m, lands; results in `runs\diagnostic\`):

```
py .\flow_diagnostic.py
```

Real-drone check before the network flies (MTF-01, PosHold, by hand):

1. Set the EKF reference location in `ap_link.py` to your real flying spot (the SITL default is Canberra; it adds about 12 deg compass error in Kuching).
2. Mark the take-off spot on the floor with tape. Fly over a patterned floor in good light.
3. Hover in PosHold for 40 s: the drone should stay on the mark.

4. Move it 1 m forward with the stick, measured with a tape measure; ArduPilot's position change should also read about 1 m, in the same direction.
5. If the distance is off, calibrate `FLOW_FXSCALER` / `FLOW_FYSCALER`; if the direction is off, check `FLOW_ORIENT_YAW`.

Keep real flights short (a few minutes): optical-flow position always drifts slowly over time.

Troubleshooting

Every problem below happened during development; each fix is already in the scripts.

What you see	Cause	Fix
Drone slowly slides into a wall during a long collection, but logged errors look small	Optical-flow position drifts: it is added up from small movements, so errors grow over time. ArduPilot thinks it is on the spot	Collect and train in GPS mode; use optical flow only for short tests
Drone wanders further from the start every episode	(Fixed) targets used to be picked around the drone's current position, so they random-walked	Targets are picked around a fixed home point
`ABORT ... distance from home=2.5 m` right at the start	Episodes started before ArduPilot knew which way the drone faces (`yaw alignment complete` came later)	Takeoff waits for yaw alignment before flying (`[link] holding still - waiting for EKF yaw alignment`)
Drone climbs to about 4 m after takeoff; first comparison episode much worse for the teacher; transfer efficiency above 100%	The climb overshoots and the drone used to hold whatever height it reached	After takeoff it returns to 2.0 m (`height 4.00 m - moving to 2.00 m`); the statistics script settles 10 s at home before each controller
`Arm: Need Position Estimate` (optical-flow mode)	GUIDED needs a position before arming	Scripts arm and climb in GUIDED_NOGPS, then switch to GUIDED

What you see	Cause	Fix
`Mode change to GUIDED failed: requires position` (optical-flow mode)	No rangefinder data from AirSim	Add the `Distance1` sensor to `settings.json` (One-time setup)
`EOF on TCP socket, attempting reconnect` after `reboot`	`reboot` breaks the AirSim link	Never `reboot`: stop AirSim + SITL, start AirSim, start SITL
`sensor mode: optflow` when you wanted GPS	SITL started from the optical-flow folder	Start SITL from `~/sitl_gps`; check `param show EK3_SRC1_POSXY` = 3
`ImportError: EXPERT_DESCRIPTION`	Old `pid_expert.py`	Use the observable teacher file
`[link] EKF position reset of X m removed`	ArduPilot reset its position estimate; the jump is removed so targets stay valid	Nothing; if it appears often in GPS mode, check GPS lock
`WARNING: still armed after landing timeout`	Landing took more than 60 s	Type `mode LAND` in MAVProxy
Loop above 50 ms or many overruns	PC busy	Close other programs; run nothing else during collection
Training validation loss flat around 0.1 or rising	Teacher uses information the network cannot see (old cascade PID, run 1 of the old pipeline: transfer efficiency 42%)	Use the observable teacher in `pid_expert.py`
Optical-flow mode in SITL: network and teacher both drift, safety stop after 1-2 episodes; flow_diagnostic shows distance ratio about 0.4	SITL's fake flow sensor (built from AirSim's GPS speed) under-reports movement in this setup; not caused by our code (GPS mode: ratio 0.90, holds within 0.01 m)	Do not use optical-flow mode in SITL; train and test in GPS mode, validate optical flow on the real drone

If something else fails, keep the terminal output and the `.json` files from the run folder: together they show settings, firmware, timing and results.

Numbers for the thesis (run 1, GPS mode)

Run 1 passes Stage 1: the network matches the teacher to within about 1% (transfer efficiency 98.85% position, 97.32% altitude) with 100% success. Source: `runs\run1\results\statistics_report_<date>.txt` (fair-start version) and the other files named below.

Item	Value	Source file
Firmware	ArduCopter 4.4.0-dev, SITL + AirSim	Collection `.json`
Dataset	60,000 samples, 200 episodes x 300 steps at 20 Hz	Collection `.json`
Split	By episode: 54,000 train / 6,000 validation (20 episodes)	`model_info.json`
Best validation loss	0.0002 (old teacher: about 0.135)	`model_info.json`, loss curve
Test success rate	Fill in from step 3	`test_hover_<date>.json`

Teacher vs network, same 10 targets (seed 42), both starting settled at home:

Metric	Teacher	Network	Gap
Position error RMSE (m)	0.344	0.348	+0.004
Altitude RMSE (m)	0.083	0.085	+0.002
Max horizontal distance (m)	1.103	1.102	-0.001
End error, last 1 s (m)	0.011	0.036	+0.025
Success rate (%)	100	100	0
Settled rate (%)	100	100	0
Transfer efficiency, position		**98.85%**	
Transfer efficiency, altitude		97.32%	
Loop time (ms)	50.0	50.0	

Key findings to report:

1. **The teacher must be observable.** The first teacher (cascade PID with integral and derivative memory) gave 42.45% transfer efficiency (position) and 32.74% (altitude), with validation loss stuck near 0.135. The network cannot see the PID's internal memory, so it cannot copy it. A teacher that decides only from what the network sees (distance to target) gave 98.85%.
2. **Fair comparison matters.** Unequal starting conditions (takeoff overshoot to 4.0 m) gave an impossible 112% / 146%. With both controllers starting settled at home, the numbers fell to a realistic 98.85% / 97.32%.
3. **Sensor choice for training.** Optical-flow position drifts over long sessions, which made long data collection fail; GPS mode gave clean data. The network input is the same 13 numbers for both, so the trained network runs unchanged with optical flow (to be shown on the real drone; the SITL optical-flow model gave only 39% of the true displacement, so it was not used for validation).
4. **Remaining gap.** When holding still, the network sits about 3-4 cm off the spot versus about 1 cm for the teacher: small, consistent, and well inside the 0.3 m settled limit. Stage 2 (PPO) starts from this network (`stage1_ppo_init.zip`).